

Идентификация процессов

Система идентифицирует процессы по уникальному числовому идентификатору - **PID** (*Process ID*). В GNU/Linux процессы образуют иерархию (дерево процессов). Корнем дерева выступает системный процесс **init** или **systemd** (в зависимости от используемой системы инициализации), запускаемый на этапе загрузки ОС. Для каждого процесса хранится идентификатор родительского процесса - **PPID** (*Parent Process ID*). Для корневого процесса значение **PPID** равно 0.

Получение общих сведений о запущенных процессах

Команда **ps** (*process status*)

Запуск **ps** без аргументов выводит процессы текущего пользователя, связанные с текущим терминалом.

Часто используемые параметры (BSD-стиль, указываются без -)

- **a** - вывести процессы всех пользователей, имеющие управляющий терминал;
- **x** - вывести процессы без управляющего терминала (и/или с терминалом, отличным от текущего);
- **u** - расширенный формат вывода (включая пользователя, время запуска и др.).

Обозначения состояний процессов (колонок **STAT**)

- **R** - выполняется или готов к выполнению;
- **S** - ожидает событие (прерываемое ожидание);
- **D** - ожидает операции ввода/вывода (непрерываемое ожидание);
- **I** - простаивающий поток ядра (*idle*);
- **Z** - процесс-зомби (*zombie*);
- **T** - остановлен (например, сигналом управления заданиями).

Команда **pstree**

Команда **pstree** отображает процессы в виде дерева, что позволяет наглядно анализировать связи «родитель-потомок».

Часто используемые параметры

- **-p** - вывести PID процессов;
- **-c** - развернуть все ветви;
- **-u** - вывести имя пользователя, запустившего процесс.

Команда **top**

top - интерактивная утилита мониторинга процессов в режиме реального времени (управление осуществляется с клавиатуры). Справка доступна по клавише **h**.

Наиболее полезные клавиши в **top**

- **M** - сортировка по потреблению памяти (%MEM);
- **P** - сортировка по загрузке CPU (%CPU , сортировка по умолчанию);
- **U** - фильтрация по пользователю (вводится имя пользователя, не UID; пустой ввод - показать все процессы);
- **i** - переключение режима отображения: все процессы / только активные (в частности, со статусом R).

Получение детальных сведений о процессах (псевдофайловая система **/proc**)

/proc - псевдофайловая система, предоставляющая интерфейс к структурам данных ядра. Большинство файлов доступно только для чтения, однако часть параметров ядра может изменяться через специальные интерфейсы.

Каждому запущенному процессу соответствует каталог **/proc/<PID>/** . Ниже перечислены наиболее часто используемые псевдофайлы/каталоги для мониторинга.

Важно: доступ к части файлов ограничен правами (как правило, требуется тот же пользователь или **root**).

- **cmdline** - командная строка запуска процесса.
- **cwd** - символьная ссылка на текущий рабочий каталог процесса.
- **environ** - переменные окружения (разделены нулевыми символами **\0**).
- **exe** - символьная ссылка на исполняемый файл процесса.
- **fd/** - каталог файловых дескрипторов процесса (символьные ссылки). Например: **0** - stdin, **1** - stdout, **2** - stderr.
- **io** - статистика ввода/вывода (объемы чтения/записи в хранилище и др.).
- **maps** - карта отображений памяти процесса и права доступа.
- **sched** - параметры планирования (в т. ч. **sum_exec_runtime** , **nr_switches**), используемые планировщиком CFS.
- **stat** - машиночитаемые сведения о процессе (набор полей).

- **status** - человекочитаемое представление основной информации из `stat`.
- **statm** - сведения о памяти в страницах. Поля:
 - **size** - общий размер адресного пространства;
 - **resident** - резидентная часть;
 - **shared** - количество разделяемых страниц;
 - **text** - код (text segment);
 - **lib** - библиотеки (не используется начиная с ядра 2.6);
 - **data** - данные + стек;
 - **dt** - «грязные» страницы (не используется начиная с ядра 2.6).

Трассировка системных вызовов

`strace` - утилита командной строки для трассировки системных вызовов и сигналов процесса. Инструмент позволяет анализировать взаимодействие программы с ядром ОС, что применимо для диагностики, отладки и исследования поведения приложений.

Типовой формат запуска:

```
strace [опции] <команда> [аргументы]
```

Наиболее используемые параметры:

- **-c** - вывести статистику системных вызовов (количество, время, ошибки);
- **-f** - трассировать дочерние процессы (наследование при `fork/clone`);
- **-e trace=SET** - фильтрация по классам/именам вызовов (например, `-e trace=file` - только файловые операции).

Обработка данных о процессах

Обработка данных о процессах обычно выполняется с использованием конвейеров команд обработки текстовых потоков и/или циклической обработки строк файлов. Рекомендуется применять утилиты **grep**, **sed**, **awk**, **tr**, **sort**, **uniq**, **wc**, **paste**, а также стандартные средства работы со строками выбранного языка программирования.

Задание на лабораторную работу

Для выполнения заданий лабораторной работы допускается использование Bash

1. Напишите скрипт, выводящий PID всех процессов, состояние которых относится к множеству {R, S, D, Z, T}.
2. Напишите скрипт, выводящий PID процесса с наибольшим временем непрерывной работы (наиболее «долго работающего»).
3. Напишите скрипт, выводящий PID последнего запущенного процесса.
4. Создайте процесс командой `sleep 10000 &`.
 1. Определите PID созданного процесса и перейдите в каталог `/proc/<PID>/`.
 2. Выведите последние 5 строк стека вызовов процесса.
 3. Выведите поля `Name`, `PPid`, `Kthread`, а также количество потоков процесса.
 4. Объясните, почему значения `Tgid` и `Pid` совпадают (для данного случая).
5. Напишите скрипт, завершающий все процессы, которые работают менее (n) секунд, где (n) задаётся параметром. Запишите в файл `killed.log` PID завершённых процессов (по одному PID в строке).
6. Выведите PID процессов и суммарное потребление оперативной памяти; отсортируйте вывод по потреблению памяти.
7. Имеется бинарный файл `troubleshooting`, завершающийся с кодом 1 без вывода диагностических сообщений. Для выполнения задания используйте `strace`. (Учтите, что программа завершается через 20 секунд)
 1. Задание считается выполненным, если программа завершится с кодом 0 и выведет сообщение `You passed`.
 2. С помощью `strace` выведите сетевые и файловые операции в файлы `troubleshooting_network.log` и `troubleshooting_file.log` соответственно (используйте параметр `-e trace=...`).
 3. По результатам анализа определите, какой из пользовательских сигналов - `SIGUSR1` или `SIGUSR2` - обрабатывается программой. Запустите программу в одном терминале, затем в другом отправьте процессу противоположный сигнал и проанализируйте результат. (Например, программой обрабатывается `SIGUSR1`, тогда противоположным сигналом будет `SIGUSR2`) Пункт считается выполненным, если программа выведет `You passed 7.3`.
8. Запустите бинарный файл `top_troubleshooting`. С помощью `top` определите, какая проблема возникает, и опишите, что происходит с файлом через некоторое время.
9. Для всех процессов, зарегистрированных в системе, определите среднее время непрерывного использования CPU (*CPU burst*) и сформируйте файл со строками вида `ProcessID=PID : Parent_ProcessID=PPID : Average_Running_Time=ART`. Значение `PPid` берите из `/proc/<PID>/status`. Значение `ART` вычисляйте как $(\text{sum_exec_runtime}) / (\text{nr_switches})$, используя данные из `/proc/<PID>/sched`. Отсортируйте строки по `PPID`.
10. В файле из предыдущего пункта после каждой группы записей с одинаковым `PPID` вставьте строку вида `Average_Running_Children_of_ParentID=N is M`, где `N` - `PPID`,

а `M` - среднее значение `ART` по всем процессам данного родителя.

11. Используя `/proc`, найдите процесс, которому выделено наибольшее количество оперативной памяти. Сравните результат с выводом `top`.
12. Напишите скрипт, изменяющий приоритет процессов, время выполнения которых превышает (N) секунд ((N) задаётся параметром). Для изменения приоритета используйте `renice`. Результат продемонстрируйте с помощью одной из утилит мониторинга, рассмотренных выше.